CHAPTER 1
INTRODUCTION

1.1 Background

The agricultural industry maintains essential food production capabilities which sustain the economic stability of framing communities. Plant diseases in crops lead to diminished plant health and produce lower amounts of agricultural yield. Early disease detection in plants stands as a vital requirement to achieve successful agricultural crop control. The project requirement to achieve successful agricultural crop control. The project applies Artificial Intelligence technology to identify plant disease through leaf pictures while it suggests appropriate agricultural chemical treatments

1.1.1 Domain Overview

The project focuses on developing Artificial Intelligence solutions which operate within the agricultural sector. The system applies deep learning methods together with computer vision technology to identify diseases in crops through analysis of leaf images. The system contains a recommendation module which delivers specific disease treatment instructions based on verified agricultural data.

1.1 Problem Statement

Farmers must deal with multiple challenges when they need to identify their crop diseases and determine which treatment options will work best for their situation. The process of manual identification needs specialized knowledge from experts who must spend extensive time doing their work. The wrong disease diagnosis will result in farmers using unsuitable agrochemicals for their crops. The proposed AI framework uses leaf picture analysis to identify plant disease while it recommends suitable agricultural chemicals which help farmers make better decisions about crop management

CHAPTER 2

LITERATURE REVIEW

1.1 Existing System

The current crop disease detection systems rely on machine learning and deep learning systems to detect diseases through analysis of leaf images. The filed of disease classification depends heavily on CNN-based models together with transfer learning techniques. The systems provide fertilizer and pesticieds recommendations which they create by analyzing disease detection results and agricultural information. The existing methods focus their attention on identifying disease but they fail to recommend suitable crops for agricultural production.

2.1.1 System Description

The current systems use deep learning models which include CNN and DensNet and MobileNet and EfficientNet to analyze images of crop leaves. The trained model receives the preprocesed input image which it uses to identify the correct disese category. The methods combine explainable AI systems

with recommendation systems which deliver extra data to users. The system performance depends on image quality and environmental conditions and dataset restrictions and different crop types which affect results.

1.1 Research Review

The research evaluation consists of four main areas which include deep learning systems for crop disease identification and lightweight CNN models and explainable AI systems and agricultural recommendation platforms. Research findings show that CNN-based models effectively identify plant diseases through their analysis of leaf image data. Research studies concentrate on developing fertilizer and crop recommendation systems which use soil nutrient data together with environmental characteristics. The research studies establish basic elements which will help create a unified system for detecting crop diseases and suggesting appropriate agricultural chemicals.

1.1 Gap Analysis

Most existing systems operate by separating their crop disease identification work from their agricultural recommendation functions which exist as independent systems. The prediction of suitable crops in crop recommendation datasets depends on soil and environmental factors instead of disease-specific agrochemical treatment information. The system requires a unified structure which will link crop disease identification to accurate treatment selection for particular diseases. The project intends to solve this problem through its development of a deep learning system which identifies diseases and recommends agricultural chemicals through an independent recommendation system.

CHAPTER 3

SYSTEM ANALYSIS & DESIGN

3.1 System Architecture

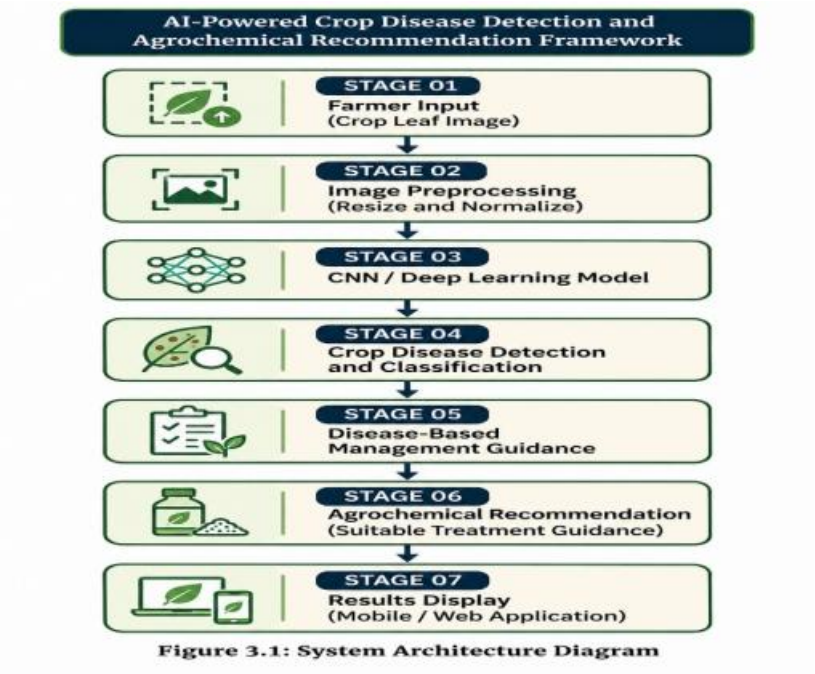


Fig.3.1. System Architecture

3.1.1 Architecture Diagram

The proposed framework consists of a frontend interface, backend processing system, deep learning model, and agrochemical recommendation module. The system receives images of crop leaves to detect diseases which it then identifies

and presents suitable treatment recommendations through its web-based platform.

3.1.2 Workflow Description

The application allows farmers to submit their crop leaf pictures by using its upload feature. The deep learning model starts its processing by receiving images which undergo size adjustment and standardization before training. The model produces disease class predictions which the recommendation system uses to find appropriate treatment guidelines from trusted agricultural sources. The application presents the final output to its users through its display system.

3.1.3 Data Flow

The frontend sends the input leaf image to the backend system for processing. The backend system prepares the image through preprocessing before it delivers the data to the disease classification model. The recommendation module uses the predicted disease to find suitable treatment details. The system sends the prediction results together with the recommendation data back to the frontend interface.

3.2 System Design

The system follows a modular design approach in which image classification and agrochemical recommendation are handled independently. This structure supports easier development, testing, and maintenance of the framework.

3.2.1 Design Approach

The frontend development process uses React with TypeScript and Vite to build the interface while Python and FastAPI serve as the backend system. The developers use TensorFlow/Keras to build deep learning models which perform disease classification and OpenCV works for image preprocessing tasks. The recommendation module offers disease-specific treatment guidance which uses verified agricultural information to help users make decisions.

3.2.2 Database Design

Recommendation module utilizes structured data that involves crop classification, disease classification, and recommendations for their treatment. The structured data can be stored in a database or knowledge base. The Crop Recommendation Dataset could have been used for crop

selection, but does not provide disease-specific agrochemical recommendations.

3.2.3 UML Diagrams

UML diagrams are used to represent the structure and behavior of the proposed system. The use-case diagram describes user interactions, the activity diagram represents the workflow, and the sequence diagram illustrates communication between the frontend, backend, classification model, and recommendation module.

3.3 Module Description

The framework is divided into three main modules:

3.3.1 Module 1 - Image Preprocessing and Disease Detection

This module accepts crop leaf images and performs preprocessing operations such as resizing and normalization. The processed image is passed to a trained deep learning model to classify the leaf as healthy or identify its disease class.

3.3.2 Module 2 - Agrochemical Recommendation

This module uses the predicted crop and disease class to retrieve suitable treatment guidance from a verified agricultural knowledge base. The recommendations are based on relevant crop, disease, and approved treatment information.

3.3.3 Module 3 - User Interface and Result Display

This module provides a user-friendly interface for uploading crop leaf images and viewing the results. It displays the predicted disease, confidence score where available, and relevant agrochemical or crop-management guidance.

CHAPTER – 4

IMPLEMENTATION

4.1 Technologies Used

The system takes a photo of a crop leaf, identifies the disease, and suggests an agrochemical treatment for it. It is written in Python, and the deep learning part is built with PyTorch.

For classification, I fine-tuned EfficientNet-B0 on the PlantVillage dataset, which has 38 classes of healthy and diseased leaves. EfficientNet-B0 is the smallest model in its family. It reaches high accuracy with relatively few parameters, so it is practical to train on limited hardware and easy to deploy later. I trained it in Google Colab for 10 epochs, and the best validation accuracy was about 99.8%.

Once the model predicts a disease, the system maps that prediction to a matching agrochemical recommendation and shows it to the user alongside the diagnosis.

4.1.1 Software Tools

Software Tool	**Purpose**
Python	Primary programming language used for implementing the system.
Google Colab / Jupyter Notebook	Used for developing, executing, and testing the model training pipeline.
Visual Studio Code	Used for Python development and project code management.
PyTorch	Used to implement, train, and evaluate the deep learning model.
Torchvision	Used for image transformations, dataset handling, and pretrained model loading.

OpenCV	Used for image processing operations where required.
NumPy	Used for numerical computations and array manipulation.
Matplotlib	Used for visualizing training performance and experimental results.
Pandas	Used for handling structured data and analysis.

4.1.2 Hardware Requirements

Training was done on Google Colab, which provides a cloud GPU without needing a dedicated machine. Fine-tuning EfficientNet-B0 on the PlantVillage dataset, which contains tens of thousands of leaf images, is much faster on a GPU than on a CPU alone. EfficientNet-B0 is also a small model, so it fits comfortably within the memory of a standard GPU.

Component	Requirement
Processor	Intel Core i5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 10 GB of available storage
GPU	NVIDIA GPU with CUDA support (recommended for faster training)
Operating System	Windows 10/11 or Linux
Input Device	Camera or image-upload facility
Display	Standard monitor or laptop display

4.1.3 Frameworks / Libraries

S.No.	Framework / Library	Description / Purpose
1	PyTorch	Used for deep learning model development, training and evaluation.
2	Torchvision	Provides pretrained models, image transformations and dataset utilities.
3	Timm	Provides pretrained computer vision models and architectures.
4	NumPy	Used for numerical computations and array manipulation.
5	Pandas	Used for dataset handling and structured data analysis.
6	OpenCV	Used for image processing and image manipulation.
7	Pillow (PIL)	Used for loading, resizing and processing crop leaf images.
8	Matplotlib	Used for visualizing training accuracy, loss and performance graphs.
9	Seaborn	Used for statistical visualization and confusion matrix heatmaps.
10	Scikit-learn	Used for model evaluation, classification metrics and data splitting.
11	SciPy	Used for scientific and numerical computations.
12	Flask	Used for backend API development and model integration, if implemented.
13	Streamlit	Used for developing an interactive user interface, if implemented.
14	TensorBoard	Used for monitoring and visualizing model training.

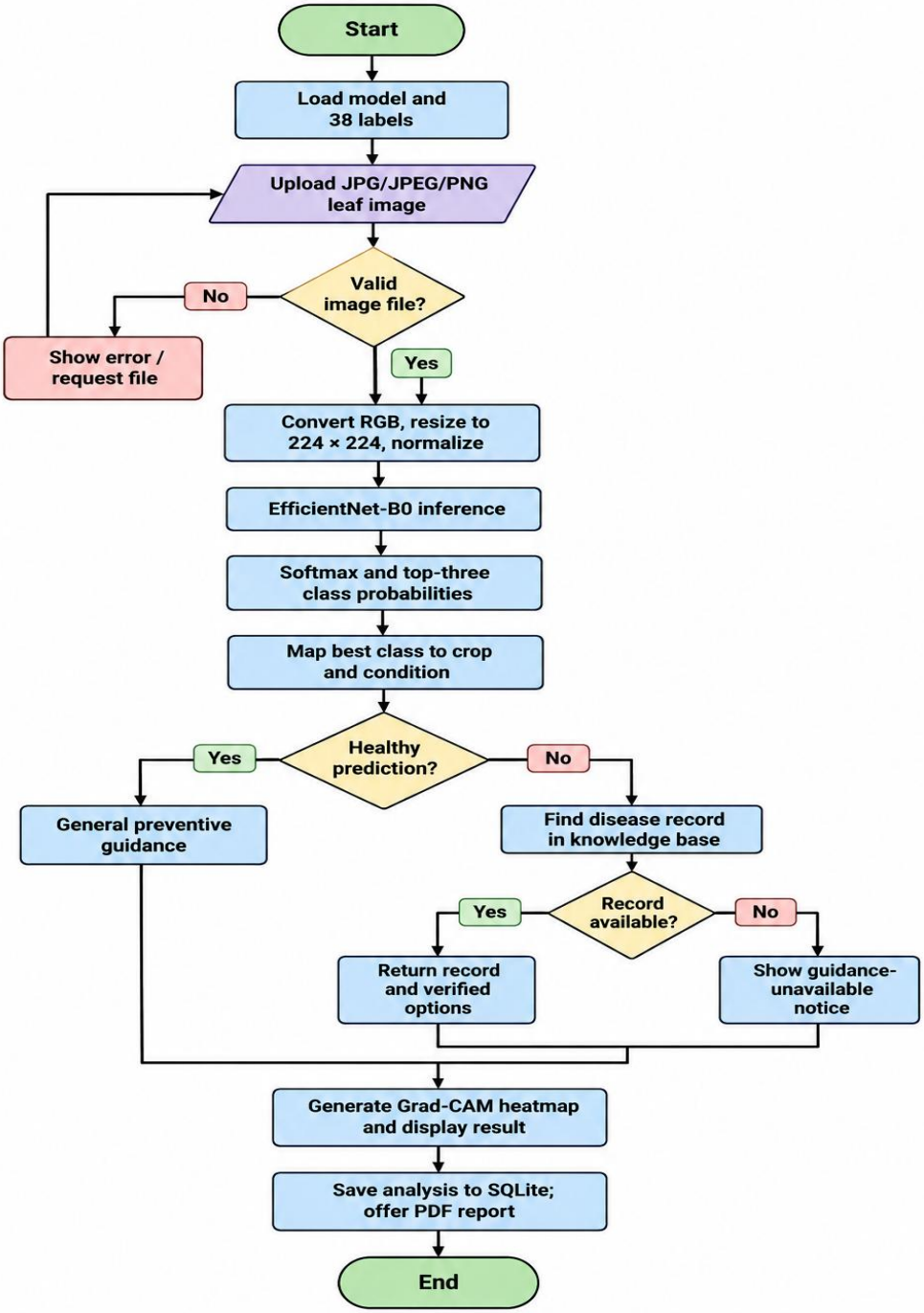
4.2 Algorithms / Models

4.2.1 Algorithm Description

1. **Loading the model.** When the application starts, it loads the trained EfficientNet-B0 checkpoint and the class mapping file. It first confirms that the mapping has exactly 38 classes, then loads the saved weights with strict matching, so a mismatch with the architecture raises an error instead of passing silently. The model runs on a GPU if CUDA is available and on the CPU otherwise.
2. **Preparing the image.** The user uploads a leaf photo in JPG, JPEG, or PNG format. The image is converted to RGB and resized to 224 × 224 pixels, then turned into a tensor. It is normalized with the standard ImageNet values: means of 0.485, 0.456, and 0.406, and standard deviations of 0.229, 0.224, and 0.225.
3. **Classifying.** The model outputs one raw score for each of the 38 classes, and softmax turns those scores into probabilities. The class with the highest probability becomes the prediction. The top three are also kept, so the user can see close alternatives.
4. **Reading the label.** Each class label combines a crop name and a condition, so the application splits it into the crop and either the disease name or “healthy”.
5. **Finding management guidance.** Healthy predictions are handled separately, since no treatment is needed. For a disease, the recommendation engine looks it up in the knowledge base, first by exact name and then by a normalized version of the name. Chemical options are shown only when the knowledge-base entry is marked as verified. If no record exists for that disease, the application says verified guidance is not available. It does not present unverified advice as fact.
6. **Explaining the result.** To show what the model focused on, Grad-CAM takes the predicted class and the activations from the final feature layer and builds a heatmap. The results screen displays the prediction, the confidence score, the top three alternatives, and the recommendation details. When available, it also shows the heatmap and its overlay on the original leaf.

7. Saving and exporting. Each prediction summary is stored in a SQLite database, so earlier results can be looked up later. The user can also generate a PDF report that contains the screening result along with any supporting information available for that prediction, such as the recommendation details.

4.2.2 Flowchart



4.2.3 Mathematical Model

The system sorts a leaf photo into one of 38 classes with EfficientNet-B0 and then looks up advice for the disease it finds.

1. Image Preprocessing

Let I be the uploaded crop leaf image. It is resized to 224×224 pixels, converted to RGB, and normalized channel-wise using the ImageNet statistics of the pretrained model:

$$X_c(u,v) = \frac{\frac{I_c(u,v)}{255} - \mu_c}{\sigma_c}, \qquad c \in \{R, G, B\}$$

$$\mu = (0.485,\ 0.456,\ 0.406), \qquad \sigma = (0.229,\ 0.224,\ 0.225)$$

The resulting input tensor is:

$$X \in \mathbb{R}^{1\times3\times224\times224}$$

2. Feature Extraction and Classification

The preprocessed image is passed through the trained EfficientNet-B0 network to produce 38 class scores (logits):

$$\mathbf{z} = f_\theta(X) = (z_1, z_2, \ldots, z_{38}) \in \mathbb{R}^{38}$$

where f_θ is the trained network and θ denotes its learned parameters.

3. Softmax Probability and Prediction

The Softmax function converts the logits into class probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{38} e^{z_j}}, \qquad i = 1,2,\ldots,38$$

where $p_i \geq 0$ and $\sum_{i=1}^{38} p_i = 1$.

The predicted class is:

$$\hat{y} = \arg\max_{i\in\{1,2,\ldots,38\}} p_i$$

The prediction confidence (in percent) is:

$$C = 100 \times \max_{1 \le i \le 38} p_i$$

The system also reports the three classes with the highest probabilities, i.e. the classes i_1, i_2, i_3 such that:

$$p_{i_1} \geq p_{i_2} \geq p_{i_3} \geq p_j \quad \text{for all other classes} j$$

4. Grad-CAM Visualization

Let $A^k \in \mathbb{R}^{U \times V}$ be the k-th feature map of the last convolutional layer, and y^c the (pre-softmax) score of class c. The importance weight of each feature map is the global average of the gradients:

$$\alpha_k^c = \frac{1}{Z} \sum_{u=1}^{U} \sum_{v=1}^{V} \frac{\partial y^c}{\partial A_{uv}^k}, \qquad Z = U \times V$$

The class activation heatmap is:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c \; A^k \right)$$

The heatmap is then upsampled to 224×224, normalized to $[0,1]$, and overlaid on the original leaf image:

$$\tilde{L}^c = \frac{L^c - \min(L^c)}{\max(L^c) - \min(L^c)}$$

5. Agrochemical Recommendation

The predicted disease class is used to retrieve disease-management information and verified agrochemical recommendations from the agricultural knowledge base:

$$R = g(\hat{y}, K)$$

where R is the generated recommendation, $\hat{y}$ is the predicted disease, K is the agricultural knowledge base, and g is the recommendation retrieval function.

4.3 Working / Screens

4.3.1 Input Processing

The user uploads a leaf photo in JPG, JPEG, or PNG format on the Disease Detection screen. They can also add optional details about the crop, such as its growth stage, how severe the disease looks, the location, the soil condition, the temperature, and the humidity.

Before classification, the system checks that the file is a valid image. It then converts the image to RGB, resizes it to 224 × 224 pixels, turns it into a tensor, and normalizes the pixel values. If the file is invalid or corrupted, the system shows an error message instead of running the model.

4.3.2 Execution Process

The preprocessed image goes to the trained EfficientNet-B0 model, which gives a probability for each of the 38 categories. The system keeps the three most likely predictions.

The top prediction is matched to its crop and disease. The system then pulls the symptoms, causes, and management advice for that disease from the knowledge base. Agrochemical options appear only when they are verified.

A Grad-CAM heatmap is also generated to show which parts of the leaf the model relied on most. If the model's confidence falls below 40%, the system displays a warning. Details of each analysis are saved in an SQLite database so they can be reviewed later.

4.3.3 Output Generation

Once classification finishes, the application shows the predicted disease, the confidence percentage, the top three predictions, and the Grad-CAM visualization. Any disease-

management advice and verified agrochemical recommendations available for that disease appear alongside them.

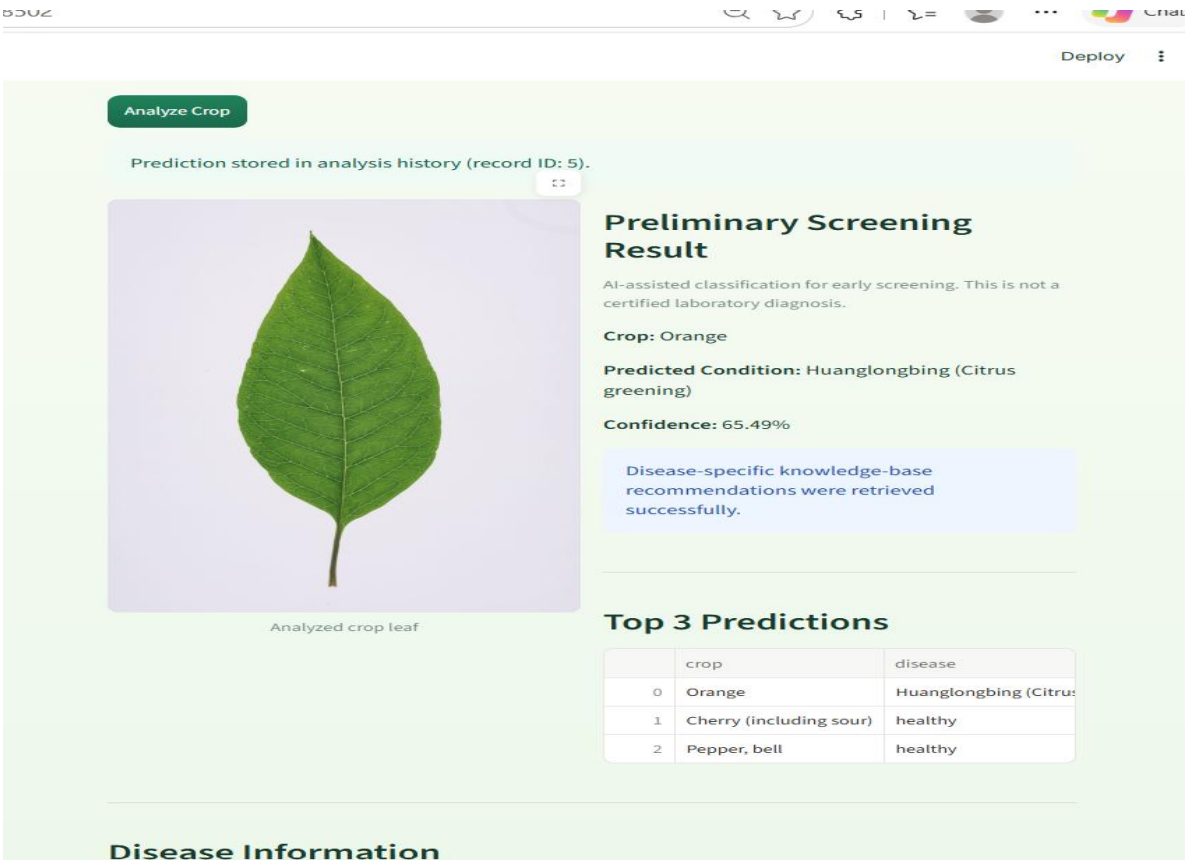
Each analysis is saved in the SQLite database. Users can go back through their prediction history and create PDF reports from within the application.

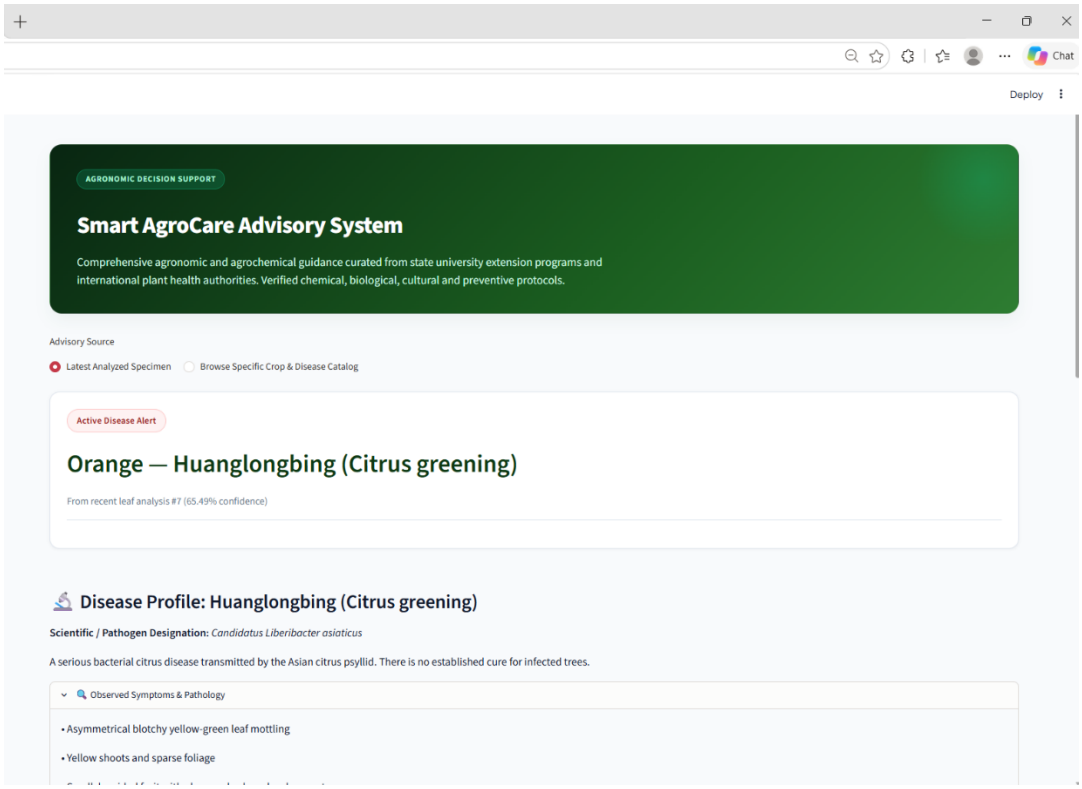
CHAPTER-5

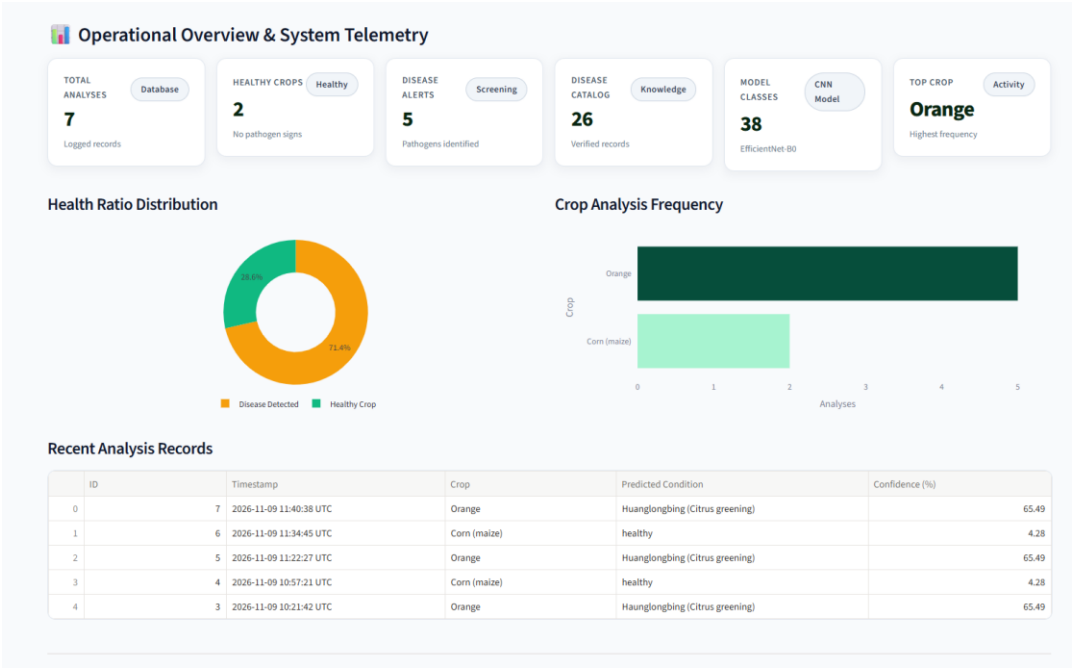
Results and Discussion

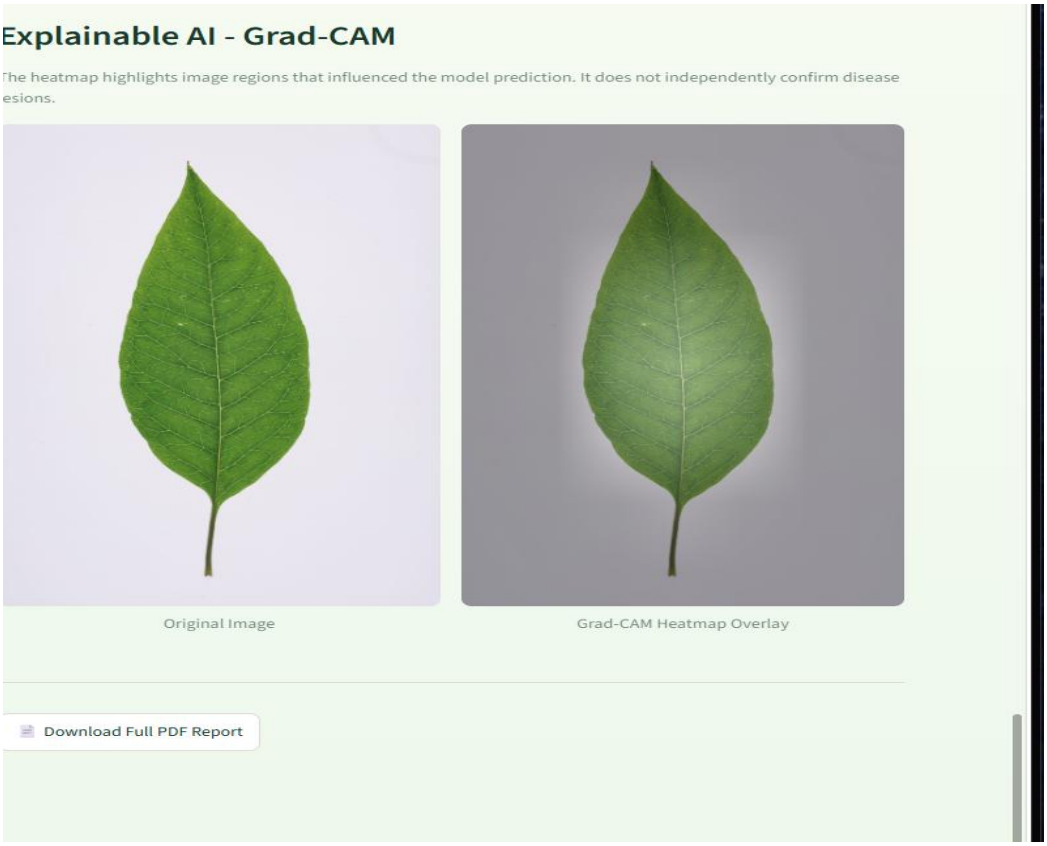
5.1 Outputs

5.1.1 Screenshots

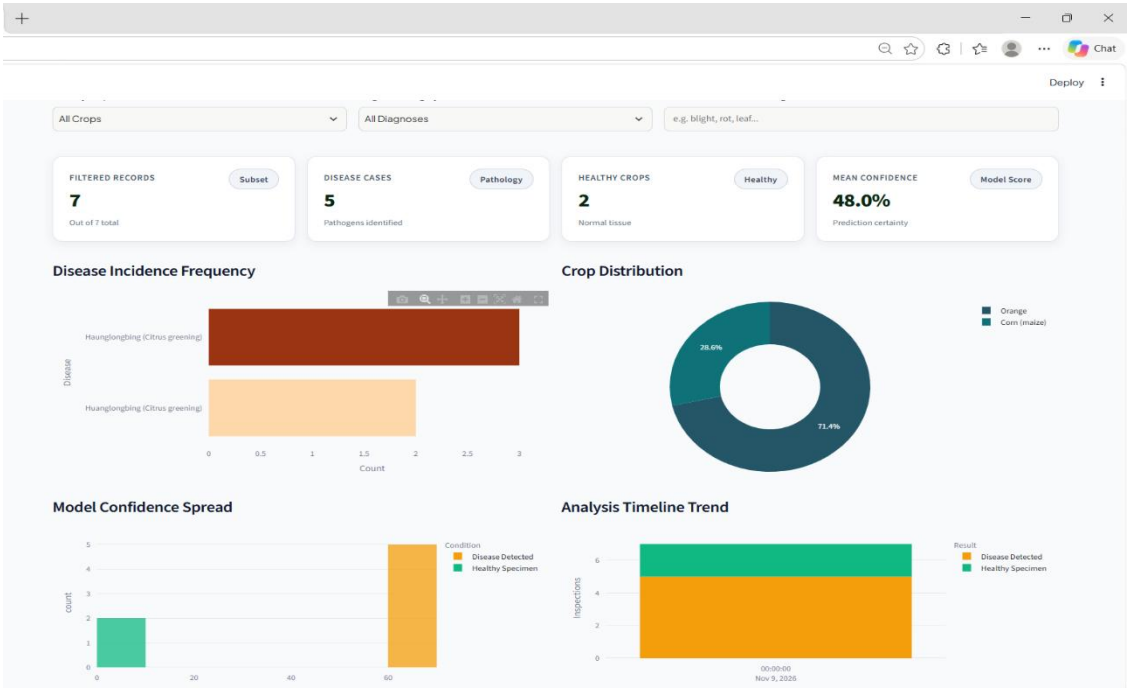



Chat
Deploy
AGRONOMIC DECISION SUPPORT
Smart AgroCare Advisory System
Comprehensive agronomic and agrochemical guidance curated from state university extension programs and international plant health authorities. Verified chemical, biological, cultural and preventive protocols.
Advisory Source
Latest Analyzed Specimen
Browse Specific Crop & Disease Catalog
Active Disease Alert
Orange — Huanglongbing (Citrus greening)
From recent leaf analysis #7 (65.49% confidence)
Disease Profile: Huanglongbing (Citrus greening)
Scientific / Pathogen Designation: Candidatus Liberibacter asiaticus
A serious bacterial citrus disease transmitted by the Asian citrus psyllid. There is no established cure for infected trees.
Observed Symptoms & Pathology
• Asymmetrical blotchy yellow-green leaf mottling
• Yellow shoots and sparse foliage


Operational Overview & System Telemetry
TOTAL ANALYSES
Database
7
Logged records
HEALTHY CROPS
Healthy
2
No pathogen signs
DISEASE ALERTS
Screening
5
Pathogens identified
DISEASE CATALOG
Knowledge
26
Verified records
MODEL CLASSES
CNN Model
38
EfficientNet-B0
TOP CROP
Activity
Orange
Highest frequency
Health Ratio Distribution
28.6%
71.4%
Disease Detected
Healthy Crop
Crop Analysis Frequency
Orange
Corn (maize)
Crop
0
1
2
3
4
5
Analyses
Recent Analysis Records
ID
Timestamp
Crop
Predicted Condition
Confidence (%)
0 7 2026-11-09 11:40:38 UTC Orange Huanglongbing (Citrus greening) 65.49
1 6 2026-11-09 11:34:45 UTC Corn (maize) healthy 4.28
2 5 2026-11-09 11:22:27 UTC Orange Huanglongbing (Citrus greening) 65.49
3 4 2026-11-09 10:57:21 UTC Corn (maize) healthy 4.28
4 3 2026-11-09 10:21:42 UTC Orange Haunglongbing (Citrus greening) 65.49



5.1.2 Graphical Results



5.1.3 Outputs

When I opened the local SQLite database to inspect it, it held seven saved analysis records. A summary of these records is given below.

Output Parameter	Observed Value
Total saved analyses	7
Orange records	5
Corn records	2
Healthy predictions	2
Disease predictions	5
Mean recorded confidence	48.00%
Predictions below 40% confidence	2

For every analysis, the application shows the prediction summary and any recommendations available for it. The SQLite database stores the uploaded filename, crop name, predicted condition, confidence score, top predictions, recommendation summary, and timestamp.

The PDF report puts the prediction and its supporting information in one document, so the user can refer back to it later.

5.2 PERFORMANCE ANALYSIS

Performance analysis shows how well the system works and how far it can be trusted. I tested the AGROVISION AI application with functional checks on each of its main parts: image preprocessing, class-label handling, prediction output, recommendation retrieval, Grad-CAM visualization, database operations, and PDF report generation.

These checks confirm that the application's features work as intended. They do not measure how accurate the classifier is, because it has not yet been evaluated on a separate labeled test dataset. For that reason, the confidence scores and the saved database records are not treated

as evidence of predictive performance. A proper evaluation on held-out test data is left for future work.

5.2.1 Metrics

Performance Metrics

To judge how well the EfficientNet-B0 model classifies crop diseases, six metrics are commonly used:

- Accuracy: the share of all evaluated images that the model classified correctly.
- Precision: of the images the model labeled as a given class, the share that truly belong to it.
- Recall: of the images that truly belong to a given class, the share the model found.
- F1-score: the harmonic mean of precision and recall, giving one balanced figure.
- Top-3 accuracy: the share of images where the true class is among the model's three highest-ranked predictions.
- Inference time: how long the trained model takes to process one image and produce a prediction.

5.2.2 EVALUATION RESULTS

The proposed AGROVISION AI system uses EfficientNet-B0 to classify crop-health conditions across 38 supported classes. For each uploaded image, the application returns a prediction, a confidence score, and up to three candidate predictions.

When I inspected the local SQLite database, it held seven analysis records. The observed results are summarized below.

Evaluation Parameter	Observed Result
Deep Learning Model	**EfficientNet-B0**
Number of Supported Classes	**38**
Maximum Predictions Displayed	**3**
Total Saved Analyses	**7**
Orange Records	**5**
Corn Records	**2**
Healthy Predictions	**2**
Disease Predictions	**5**

5.2.3 OBSERVATIONS

During development and functional testing of AGROVISION AI, the following was observed:

1. The system covers the whole workflow in one place: uploading a leaf image, preprocessing it, predicting, and showing the result.
2. EfficientNet-B0 classifies images across 38 crop-health classes.
3. Up to three candidate predictions are shown, so users can see close alternatives to the top result.
4. Grad-CAM highlights the parts of the leaf that most influenced the prediction.
5. The recommendation module pulls disease descriptions, symptoms, causes, and available management advice from the agricultural knowledge base.
6. When confidence falls below 40%, a warning tells the user to interpret the result with caution.
7. Each analysis is stored in the SQLite database, so earlier results can be reviewed.
8. An analytics dashboard summarizes the saved predictions and crop distribution in charts.

9. The PDF report generation feature provides a convenient way to save and share analysis results.

The observations demonstrate the functional capabilities of the developed application. However, further testing with verified field images and labeled datasets is necessary to establish its predictive reliability.

5.3 COMPARISON

5.3.1 COMPARISON WITH EXISTING SYSTEMS

Comparison Parameters	**Traditional Manual Identification**	**Basic Image-Based Detection Systems**	**Proposed AGROVISION AI**
Disease Identification	Performed through manual observation and expert consultation.	Uses image processing or machine learning techniques.	Uses EfficientNet-B0 deep learning model for image classification.
Image-Based Analysis	Depends on human visual inspection.	Accepts crop leaf images for classification.	Accepts uploaded leaf images and performs automated analysis.
Classification Capability	Depends on the knowledge and experience of the person.	Depends on the trained model and supported disease categories.	Supports classification across 38 crop disease and healthy-condition classes.
Disease Information	Obtained through manual consultation or external sources.	May provide only the predicted disease name.	Retrieves disease descriptions, symptoms, and causes from a structured knowledge base.

Recommendation System	Recommendations are obtained from agricultural experts.	Often limited to disease identification.	Provides available chemical, biological, and cultural management guidance along with preventive measures.
Explainability	Based on human observation.	May not provide visual explanations of predictions.	Uses Grad-CAM to highlight image regions influencing the model prediction.
User Interface	Generally involves direct communication with experts or manual inspection.	Varies depending on the application.	Provides an interactive Streamlit-based web interface.
Analysis History	Usually maintained manually.	Depends on the system.	Supports storing and retrieving previous analysis records using SQLite.
Report Generation	Requires manual documentation.	Depends on the system.	Supports PDF report generation.
Data Visualization	Generally depends on manual records.	Limited or application-dependent.	Provides an interactive analytics dashboard using Plotly.

5.3.2 ADVANTAGES OF PROPOSED SYSTEM

1. **Automated disease detection:** EfficientNet-B0 identifies the crop's health condition from an uploaded leaf image.
2. **Multiple predictions:** The system covers 38 crop-health classes and shows up to three likely predictions for each image.

3. **Visual explanation:** Grad-CAM marks the regions of the leaf that most influenced the result.
4. **Agricultural knowledge integration:** A structured knowledge base supplies disease descriptions, symptoms, causes, and available management advice.
5. **PDF Report Generation:** Allows users to generate and download reports containing prediction details and supporting information.
6. **User-friendly interface:** A Streamlit interface lets users upload images, get predictions, and view the results in one place.
7. **Low-confidence warning:** When a prediction falls below the set confidence threshold, the system warns the user to read the result carefully.

5.3.3 LIMITATIONS

Despite its useful features, the system has several limitations:

1. **Limited evaluation evidence:** Accuracy, precision, recall, and F1-score have not yet been measured on a separate labeled test dataset.
2. **Confidence score reliability:** A high softmax confidence does not guarantee a correct prediction, and low-confidence results should be read with caution.
3. **Limited class coverage:** The model only knows its 38 classes, so crops and diseases outside them may be classified wrongly.
4. **Dependence on image quality:** Poor lighting, low resolution, cluttered backgrounds, and unclear leaves can reduce prediction quality.
5. **Real-world field variation:** Photos taken in actual fields can differ from the training images, which may affect how well the model performs.

CHAPTER-6

CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

AGROVISION AI is an AI-powered system for crop disease detection and agrochemical recommendation. It helps users identify crop diseases from leaf photos and find relevant disease-management information.

At its core, EfficientNet-B0 analyzes an uploaded leaf image and sorts it into one of 38 disease or healthy classes. Automating this step reduces the need for manual visual inspection and makes a first check of crop health easier to get.

Beyond classification, a knowledge-based recommendation engine retrieves disease descriptions, symptoms, causes, management practices, and preventive measures from a structured agricultural knowledge base. Grad-CAM adds a visual explanation by highlighting the regions of the leaf that shaped the prediction.

The application is built with Python and Streamlit and offers an interactive interface for uploading images, detecting diseases, viewing recommendations, and managing past analyses. SQLite stores the analysis history, Plotly charts the recorded data, and the system can also produce PDF reports.

Overall, AGROVISION AI brings together deep learning, knowledge retrieval, explainable AI, and interactive visualization in one decision-support tool. It shows how AI can support crop health monitoring and give users quick access to disease-management guidance.

6.1.1 SUMMARY OF WORK

The main activities carried out during the project were:

1. **Requirement analysis:** I studied the difficulties of identifying crop diseases and found a need for an automated system that detects diseases from images and recommends what to do.
2. **Dataset and class preparation:** I set up the classification configuration for the 38 disease and healthy-condition classes.
3. **Model implementation:** I integrated the trained EfficientNet-B0 model into the application using PyTorch.
4. **Image preprocessing:** I added image validation, RGB conversion, resizing, tensor conversion, and normalization so uploaded images are ready for the model.
5. **Disease classification:** I built the prediction module, which returns the predicted crop and condition along with a confidence score.
6. **Recommendation engine:** I built a knowledge-based engine on structured JSON data that retrieves disease descriptions, symptoms, causes, management information, and preventive measures.
7. **Explainable AI:** I integrated Grad-CAM to produce heatmaps and overlays showing which image regions influenced the prediction.
8. **Application development:** I built a Streamlit application with screens for the dashboard, disease detection, recommendations, analytics, history, and information.
9. **Database integration:** I used SQLite to store and retrieve past analysis records.
10. **Report generation:** I added PDF report generation so users can download a record of each analysis.

6.1.2 ADVANTAGES

1. **Automated disease identification:** A trained deep learning model classifies leaf images and flags likely disease conditions.
2. **Time-efficient analysis:** Automatic classification cuts down on repeated manual inspection.
3. **Deep learning-based classification:** EfficientNet-B0 extracts visual features and sorts images into 38 disease and healthy classes.
4. **Integrated recommendations:** A structured agricultural knowledge base supplies disease information and available management advice.

5. **Explainable AI support:** Grad-CAM highlights the image regions that influenced the prediction.
6. **User-friendly interface:** The Streamlit application lets users upload images and see the analysis results without any technical setup.
7. **Access to disease information:** Users can read disease descriptions, symptoms, causes, and preventive measures.
8. **Analysis history:** An SQLite database keeps earlier analyses so users can look them up again.
9. **PDF report generation:** Users can create and download a report of each analysis.
10. **Interactive analytics:** Charts and dashboards summarize the recorded analysis data.
11. **Modular architecture:** Disease detection, image preprocessing, recommendation generation, database operations, and the user interface are kept separate, which makes the application easier to maintain and extend.

6.1.3 APPLICATIONS

1. Agricultural field support: Farmers can upload photos of affected leaves for a first screening of crop disease.
2. Crop health monitoring: Image-based classification can be used to keep track of crop health.
3. Agricultural advisory support: The system gives disease information and available management advice to help with farming decisions.
4. Educational institutions: Students and researchers can use it to learn how deep learning and computer vision are applied in smart agriculture.
5. Agricultural research: It can serve as a prototype for further work on crop disease classification, explainable AI, and knowledge-based agricultural systems.
6. Agricultural extension activities: Extension workers and organizations can use it to demonstrate AI-based disease identification.
7. Smart agriculture applications: It can be a starting point for adding AI to future digital agriculture platforms.

8. Crop disease awareness: Users can learn about common crop diseases, their symptoms, causes, and preventive measures.

6.1.4 LIMITATIONS

1. Limited evaluation evidence: The model's accuracy, precision, recall, and F1-score have not yet been measured on a separate labeled test set.
2. Reliability of confidence scores: A high softmax score does not mean the prediction is right, and low-confidence results should be treated carefully.
3. Limited class coverage: The model recognizes only its 38 classes, so a crop or disease outside them may be misclassified.
4. Dependence on image quality: Poor lighting, low resolution, busy backgrounds, and unclear leaves can lower prediction quality.
5. Field variation: Real field photos can look quite different from the training images, which may affect performance.

6.2 FUTURE SCOPE

1. **More crops and diseases:** Training on a wider and more representative set of images would let the system cover additional crops and disease categories.
2. **Better model performance:** Newer architectures, transfer learning approaches, and model optimization could improve accuracy and generalization.
3. **Real-world image data:** Adding photos taken directly in fields would help the model handle varied lighting, backgrounds, and environmental conditions.
4. **Mobile application:** A mobile version would let farmers photograph a leaf with their phone and get a first screening result on the spot.
5. **Multilingual support:** Regional language options would make the application usable for farmers from different language backgrounds.
6. **Advanced recommendation system:** A larger knowledge base validated by experts could give more complete disease-management advice.
7. **Weather-based risk assessment:** Temperature, humidity, and rainfall data could be added to estimate the risk of disease outbreaks.

8. **IoT integration:** Sensors measuring soil moisture, temperature, and humidity could support fuller crop monitoring.
9. **Cloud deployment:** Hosting on cloud infrastructure would improve access, scalability, central data management, and availability.
10. **More explainability methods:** Other explainable AI techniques could give clearer and more detailed visual insight into the model's predictions.
11. **Expert validation and feedback:** Agricultural experts could check the disease information and recommendations, and a feedback feature could help improve the knowledge base over time.
12. **Continuous improvement:** The model could be kept current through regular dataset updates, retraining, and evaluation on newly collected images.

Reference & Bibliography

1. Tan, M., and Le, Q. V. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *Proceedings of the 36th International Conference on Machine Learning*, pp. 6105–6114.
2. Mohanty, S. P., Hughes, D. P., and Salathé, M. (2016). "Using Deep Learning for Image-Based Plant Disease Detection." *Frontiers in Plant Science*, Vol. 7, Article 1419.
3. Ferentinos, K. P. (2018). "Deep Learning Models for Plant Disease Detection and Diagnosis." *Computers and Electronics in Agriculture*, Vol. 145, pp. 311–318.
4. Barbedo, J. G. A. (2016). "A Review on the Main Challenges in Automatic Plant Disease Identification Based on Visible Range Images." *Biosystems Engineering*, Vol. 144, pp. 52–60.
5. Hughes, D. P., and Salathé, M. (2015). "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics." arXiv:1511.08060.
6. Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., and Batra, N. (2020). "PlantDoc: A Dataset for Visual Plant Disease Detection." *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, pp. 249–253.
7. Fuentes, A., Yoon, S., Kim, S. C., and Park, D. S. (2017). "A Robust Deep-Learning-Based Detector for Real-Time Tomato Plant Diseases and Pests Recognition." *Sensors*, Vol. 17, No. 9, Article 2022.
8. Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
9. Chollet, F. (2017). "Xception: Deep Learning with Depthwise Separable Convolutions." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1251–1258.
10. He, K., Zhang, X., Ren, S., and Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778.
11. PlantVillage Dataset. Available: https://plantvillage.psu.edu/
12. PyTorch Documentation. Available: https://pytorch.org/docs/
13. TensorFlow Documentation. Available: https://www.tensorflow.org/
14. Scikit-learn Documentation. Available: https://scikit-learn.org/
15. EfficientNet Documentation and Model Resources. Available: https://keras.io/api/applications/efficientnet/

Appendix A: Front Page of the Conference Paper

An AI-Powered Crop Disease Detection and Agrochemical Recommendation Framework

Chalini M
CSE–AIML
Alliance University, Bengaluru, India
mchaliniBTECH23@ced.alliance.edu.in

Spandana Basava Kulal
CSE–AIML
Alliance University, Bengaluru, India
bspandanaBTECH23@ced.alliance.edu.in

Dr. Qawsar Gulzar
Assistant Professor
Department of CSE
Alliance University, Bengaluru, India
qawsar.gulzar@alliance.edu.in

***Abstract*—Crop diseases are a major challenge in agriculture, as they can reduce crop productivity and affect the quality of agricultural output. Early and accurate disease identification is essential for supporting timely crop management decisions. Traditional disease diagnosis often relies on manual inspection, which can be time-consuming and may require specialized agricultural knowledge. This paper presents an AI-powered framework for crop disease detection and agrochemical recommendation using deep learning. The framework employs the EfficientNetB0 convolutional neural network, initialized with ImageNet-pretrained weights, to classify plant leaf images into disease categories. The model is trained and validated using the PlantVillage dataset, which contains labeled images of healthy and diseased plant leaves. Transfer learning is applied to improve feature extraction and classification performance. Following disease identification, the framework is designed to provide disease-specific agrochemical guidance through a recommendation module. The proposed approach aims to connect automated disease diagnosis with actionable agricultural information through an integrated system. The framework can support early disease identification and assist users in making informed crop-management decisions. Its effectiveness is evaluated using classification performance metrics, including accuracy, precision, recall, and F1-score. The work highlights the potential of deep learning-based crop disease detection integrated with recommendation support for precision agriculture.**

***Index Terms*—Crop disease detection, EfficientNetB0, deep learning, transfer learning, PlantVillage dataset, agrochemical recommendation, precision agriculture.**

I. Introduction

Agriculture plays a vital role in food security and economic development. However, crop diseases can significantly reduce agricultural productivity, affect crop quality, and increase production costs. Early disease detection is therefore essential for minimizing crop losses. Conventional diagnosis mainly relies on visual inspection, which can be time-consuming and requires domain expertise.

Recent advancements in deep learning have enabled automated plant disease detection using leaf images. Convolutional Neural Networks (CNNs) and architectures such as DenseNet, EfficientNet, and Vision Transformers have shown promising results in disease classification. The PlantVillage dataset is widely used for evaluating these methods. However, models trained on controlled images may face challenges in real-world conditions due to variations in lighting, backgrounds, and image quality.

Despite the progress in disease classification, identifying a disease alone may not provide sufficient guidance for effective crop management. Farmers also require appropriate treatment and prevention measures. Existing studies have explored pesticide and fertilizer recommendation systems, but integrating disease detection with reliable, disease-specific agrochemical guidance remains a challenge.

To address this gap, this paper proposes an AI-powered crop disease detection and agrochemical recommendation framework. The disease detection component uses EfficientNetB0 with transfer learning and is trained and validated using the PlantVillage dataset. The predicted disease category is then connected to an agrochemical recommendation module to support informed crop-management decisions.

The main contributions of this work are:

- Development of an EfficientNetB0-based crop disease classification component using transfer learning.
- Integration of disease classification with disease-specific agrochemical recommendation support.
- Evaluation of the disease detection model using accuracy, precision, recall, and F1-score.

The paper is organized as follows: Section II reviews related work, Sections III and IV describe the proposed framework and methodology, Section V presents the results, Section VI discusses agrochemical recommendations, and Section VII concludes the paper.

II. Related Work

Recent studies have explored deep learning techniques for automated plant disease detection. CNN-based architectures, transfer learning, and advanced models such as DenseNet and Vision Transformers have demonstrated promising results in classifying plant diseases using leaf images. The PlantVillage dataset is commonly used for evaluating these approaches. Researchers have also investigated explainable AI, disease severity estimation, and agricultural recommendation systems to improve decision-making. However, challenges such as variations in real-world conditions, computational complexity, and the integration of disease detection with reliable agrochemical recommendations remain. Table I presents a comparative analysis of selected studies, highlighting their methodologies, limitations, and relevance to the proposed framework.

Appendix B: Conference Submission Summary

Submission Summary

Conference Name
2027 10th International Conference on Machine Learning and Natural Language Processing (MLNLP 2027)

Paper ID
16

Paper Title
An AI-Powered Crop Disease Detection and Agrochemical Recommendation Framework

Abstract
Crop diseases are a major challenge in agriculture, as they can reduce crop productivity and affect the quality of agricultural output. Early and accurate disease identification is essential for supporting timely crop management decisions. Traditional disease diagnosis often relies on manual inspection, which can be time-consuming and may require specialized agricultural knowledge. This paper presents an AI-powered framework for crop disease detection and agrochemical recommendation using deep learning. The framework employs the EfficientNetB0 convolutional neural network, initialized with ImageNet-pretrained weights, to classify plant leaf images into disease categories. The model is trained and validated using the PlantVillage dataset, which contains labeled images of healthy and diseased plant leaves. Transfer learning is applied to improve feature extraction and classification performance. Following disease identification, the framework is designed to provide disease-specific agrochemical guidance through a recommendation module. The proposed approach aims to connect automated disease diagnosis with actionable agricultural information through an integrated system. Its effectiveness is evaluated using classification performance metrics, including accuracy, precision, recall, and F1-score. The work highlights the potential of deep learning-based crop disease detection integrated with recommendation support for precision agriculture.

Created
2026-11-09 18:03:16 IST

Last Modified
2026-11-09 18:03:16 IST

Authors
Spandana Basava Kulal (Alliance University) <spandanakulal0123@gmail.com>
Chalini M (Alliance University) <chalinim99@gmail.com>

Submission Files
Capstone Project Conference Paper.pdf (3.1 Mb, 2026-11-09 18:03:04 IST)

Appendix C: Source Code

```
@dataclass
class PredictionResult:
    crop: str
    disease: str
    confidence: float
    top_predictions: list
    prediction_label: str
    status: str
    message: str
    @property
    def confidence_pct(self):
        return self.confidence * 100
def build_efficientnet_b0(
    num_classes: int = EXPECTED_NUM_CLASSES) -> torch.nn.Module:
    """Create the EfficientNet-B0 architecture used during training."""
    model = efficientnet_b0(weights=None)
    in_features = model.classifier[1].in_features
    model.classifier[1] = torch.nn.Linear(
        in_features,
        num_classes)
    return model
def load_class_mapping(mapping_path=CLASS_MAPPING_PATH) -> dict:
    """Load the numeric class-to-label mapping."""
    mapping_path = Path(mapping_path)
    if not mapping_path.exists():
        raise FileNotFoundError(
            f"Class mapping file not found: {mapping_path}")
```

```
    with open(mapping_path, "r", encoding="utf-8") as file:
        mapping = json.load(file)
    if isinstance(mapping, list):
        mapping = {
            str(index): label
            for index, label in enumerate(mapping) }
    if not isinstance(mapping, dict):
        raise ValueError("Invalid class mapping format.")
    mapping = {
        str(key): str(value)
        for key, value in mapping.items() }
    if len(mapping) != EXPECTED_NUM_CLASSES:
        raise ValueError(
            f"Expected {EXPECTED_NUM_CLASSES} classes, "
            f"but found {len(mapping)}." )
    return mapping
def parse_label(label: str) -> tuple:
    """
    Convert a PlantVillage label such as
    Apple___Apple_scab into crop and disease names.
    """
    label = str(label).strip()
    if "___" in label:
        crop, disease = label.split("___", 1)
    elif "__" in label:
        crop, disease = label.split("__", 1)
    else:
        parts = label.split("_", 1)
        crop = parts[0]
```

```
        disease = parts[1] if len(parts) > 1 else "Unknown"
    crop = crop.replace("_", " ").strip()
    disease = disease.replace("_", " ").strip()
    # Canonicalize known dataset label typo (PlantVillage Haunglongbing -> Huanglongbing)
    if "haunglongbing" in disease.lower():
        import re
        disease = re.sub(r'(?i)haunglongbing', 'Huanglongbing', disease)
    return crop, disease
def validate_checkpoint_compatibility(model, model_path):
    """Load the actual trained weights from the checkpoint."""
    model_path = Path(model_path)
    if not model_path.exists():
        raise FileNotFoundError(
            f"Model checkpoint not found: {model_path}" )
    try:
        checkpoint = torch.load(
            model_path,
            map_location="cpu",
            weights_only=False)
        if not isinstance(checkpoint, dict):
            raise ValueError("Invalid checkpoint format.")
        if "model_state_dict" in checkpoint:
            state_dict = checkpoint["model_state_dict"]
        elif "state_dict" in checkpoint:
            state_dict = checkpoint["state_dict"]
        else:
            state_dict = checkpoint
        if not isinstance(state_dict, dict):
            raise ValueError("Model weights are missing.")
```

```
# Strict loading ensures that the architecture
# matches the trained EfficientNet-B0 checkpoint.
model.load_state_dict(state_dict, strict=True)
return True
```